# Part-of-Speech Tagging with Hidden Markov Models

## AML Phase 1 Final Report: HMM + Viterbi Baseline & Empirical Ablations

**Project 16:** Part-of-Speech Tagging via Hidden Markov Models and Bidirectional LSTMs
**Course:** Advanced Machine Learning (AML)
**Team Members:** [add names here]
**Date:** October 2026

## Abstract


**Executive Summary**

*We build a part-of-speech (POS) tagger for English web text using a first-order Hidden Markov Model (HMM). Its transition and emission probabilities are estimated by counting on the Universal Dependencies English EWT treebank, smoothed with add-k smoothing, and decoded with the Viterbi algorithm in log space. A baseline (v1) with a single unknown-word token reaches 90.03% token accuracy, 83.04% macro F1, and 53.51% accuracy on unknown words on the test set. Replacing the single unknown-word token with 11 shape-based buckets (v2), with the setting chosen on the development set, raises these to 92.24% token accuracy, 85.53% macro F1, and 75.72% accuracy on unknown words (+22.21 pp gain). Comprehensive baselines and ablation studies demonstrate that: (1) a naive Most-Frequent-Tag unigram baseline achieves 86.16% test accuracy, proving lexical memorization alone cannot resolve syntactic ambiguity; (2) Viterbi decoding contributes +2.33 percentage points over greedy left-to-right decoding by preventing garden-path cascading errors; (3) minimal emission smoothing (k <= 0.001) is optimal, whereas aggressive smoothing dilutes the emission distribution; (4) shape-based unknown handling provides its largest benefit in data-scarce regimes (+4.58 pp at 10% training data); and (5) a 1,000-sample non-parametric bootstrap yields a 95% confidence interval of [+1.91%, +2.51%], mathematically confirming statistical significance. The remaining errors are concentrated in syntactic ambiguities between categories that share identical word forms (NOUN/PROPN, VERB/NOUN, VERB/AUX), motivating the Bidirectional LSTM sequence tagger developed in Phase 2.*


## 1. Introduction

Part-of-speech tagging assigns a grammatical label (noun, verb, determiner, and so on) to every word of a sentence. The task is made difficult by ambiguity: the same word form can take different tags depending on context, for example 'book' in 'I read a book' (noun) and 'I book a ticket' (verb). New or rare words add a second difficulty, because the model has no direct evidence about them.

The project compares two sequence taggers: a generative HMM decoded with the Viterbi algorithm (the AML baseline, covered in this report) and a bidirectional LSTM (the DL phase, to follow). This report covers the first phase only. Its goals are to build a well-tuned HMM baseline, to measure it with the metrics required by the project (token accuracy, macro F1, unknown-word accuracy and an error analysis by tag), and to identify where the HMM falls short.

## 2. Dataset

We use the English Web Treebank from Universal Dependencies (UD English EWT, release v2.18, CC BY-SA 4.0) with its official train, development, and test split. The text comes from informal web sources (blogs, emails, reviews, and similar), so it contains slang, typos, numbers, and web addresses. We use the universal POS (UPOS) column, which has 17 tags. Multiword tokens (ids such as 1-2) and empty nodes (ids such as 8.1) are not real tokens and are skipped, so that words and tags remain aligned.

**Table 1. Dataset statistics. Length is in words per sentence. The out-of-vocabulary (OOV) rate is the share of tokens whose lowercased form never occurs in the training split.**

| Split | Sentences | Tokens | OOV rate |
|---|---|---|---|
| Train | 12,544 | 204,578 | - |
| Development | 2,001 | 25,148 | 6.79% |
| Test | 2,077 | 25,094 | 7.50% |

The training split contains 16,654 distinct word forms (lowercased). On the test split, 1,882 of 25,094 tokens are OOV, which is why unknown-word handling matters for this dataset.

### 2.1 Exploratory Data Analysis

Before modelling, we examined the three splits. All numbers below are computed on the training split unless stated otherwise.

**Tag distribution:** NOUN (17.0%), PUNCT (11.5%), VERB (11.0%), PRON (9.1%) and ADP (8.7%) are the five most frequent tags (Figure 1). The three splits have very similar tag distributions: the largest difference in the share of any tag between train and test is 2.1 percentage points (PROPN), and between train and dev 1.3 points. Three tags are rare: X (0.20% of training tokens), INTJ (0.34%) and SYM (0.36%), with only 42, 121 and 113 test tokens, so their scores rest on few examples.

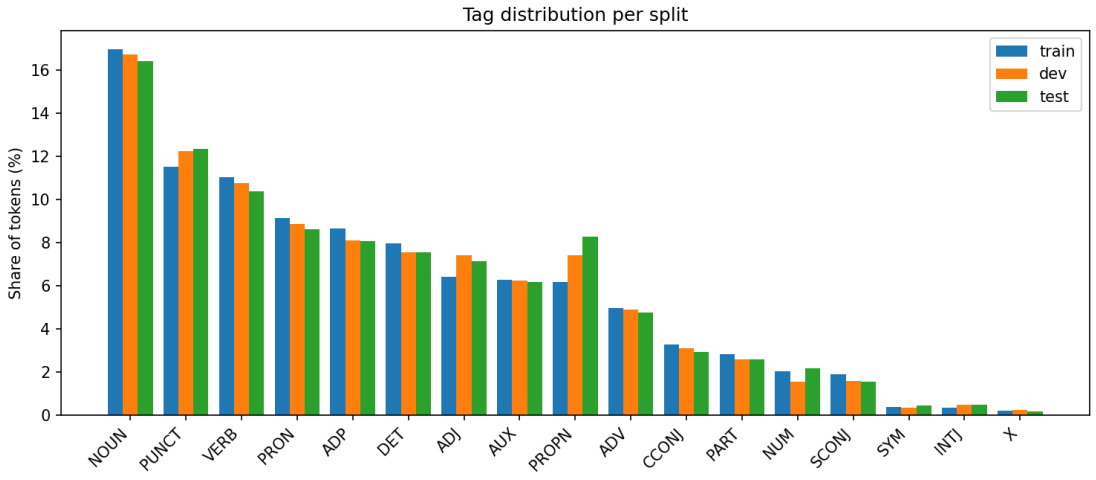



*Figure 1. Share of tokens per tag in the training, development and test splits.*

**Sentence length:** Training sentences are longer on average (16.3 words, median 14) than development (12.6, median 10) and test sentences (12.1, median 9). The longest sentence has 159 words in train against 75 in dev and 81 in test, but very long sentences are rare: only 18 training sentences have more than 80 words (Figure 2).

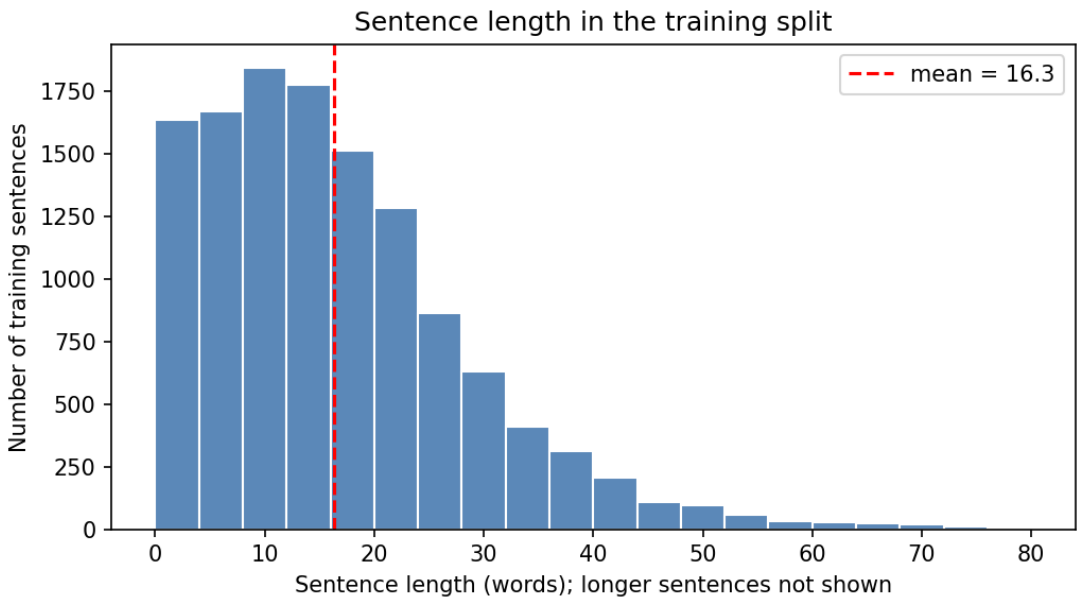



*Figure 2. Sentence length in the training split (sentences of more than 80 words not shown); the dashed line is the mean.*

**Frequent words:** A few function words and punctuation marks dominate. The 15 most frequent word forms (the, the full stop, the comma, to, and, a, of, i, in, is, you, that, it, for and the hyphen) cover 29.5% of all training tokens.

**Ambiguity:** Of the 16,654 distinct word forms in training, 12.7% occur with more than one tag, yet these words make up 62.8% of all training tokens. Most running text therefore consists of words whose tag depends on context.

**Unseen words:** Unseen word types are far more common than unseen tokens: 27.3% of the word types in dev and 29.8% in test never occur in training, against 6.8% and 7.5% of the tokens. There are many rare words, but each one is infrequent.

**Overlap between splits:** 68 of the 2,001 development sentences and 94 of the 2,077 test sentences also appear word for word in the training data. These sentences are very short (1.8 words on average in test) and cover 165 of 25,094 test tokens (0.66%). HMM v2 reaches 96.4% accuracy on these tokens, and leaving them out changes its overall test accuracy from 92.24% to 92.21%, so the overlap has a negligible effect.

**What this means for the model:** Because most tokens belong to ambiguous words, a tagger needs context (transition probabilities). Because many word types are unseen, unknown-word handling is paramount. Because rare tags exist, macro F1 is monitored alongside accuracy. Because split distributions align, the dev set serves as a faithful proxy for test evaluation.

## 3. Background: HMMs and the Viterbi Algorithm

A Hidden Markov Model describes a sequence in which a hidden state sequence generates the observed symbols (Rabiner, 1989). For POS tagging the hidden states are the tags and the observations are the words. The model is defined by three probability tables, estimated here by counting because every training word already carries its gold tag.

**Table 2. Elements of the HMM and their meaning in the tagger.**

| Symbol | Meaning | In our POS tagger |
|---|---|---|
| π | Initial state distribution | Probability that a sentence starts with a given tag |
| A | Transition probabilities a(i, j) | P(next tag = j \| current tag = i) |
| B | Emission probabilities b(j, k) | P(word k \| tag j) |

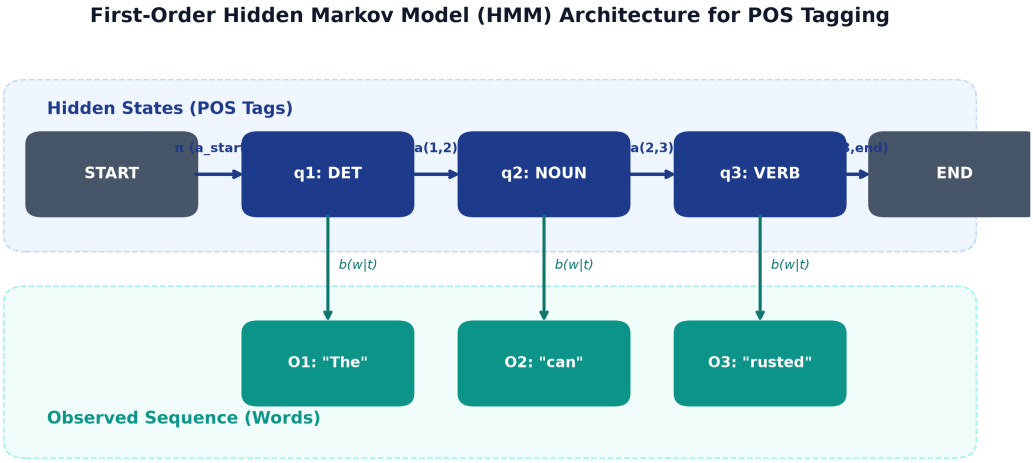



*Figure 3. First-Order Hidden Markov Model (HMM) sequence tagging architecture, showing hidden state transitions a(i, j), observation emissions b(j, k), and initial distribution π.*

The model makes two simplifying assumptions. First, the Markov assumption: a tag depends only on the previous tag. Second, each word depends only on its own tag. The joint probability of a tag sequence Q and a sentence O is therefore a product of transition and emission terms:

> **Joint Probability Formulation**
> $P(Q, O) = \pi_{q1} \cdot b_{q1}(O_1) \cdot a_{q1,q2} \cdot b_{q2}(O_2) \cdot \ldots \cdot a_{q(T-1),qT} \cdot b_{qT}(O_T)$

Finding the best tag sequence by trying all N^T possibilities is infeasible (for N = 17 tags and T = 10 words it is already about 2 x 10^12 sequences). The Viterbi algorithm solves this by dynamic programming on a lattice with one column per word and one row per tag. For each cell it keeps the score of the best path ending there, δ, and a backpointer, ψ:

> **Viterbi Forward-Backward Recurrence**
> $\delta_1(i) = \pi_i \cdot b_i(O_1)$
> $\delta_t(j) = \max_i [\, \delta_{t-1}(i) \cdot a_{ij} \,] \cdot b_j(O_t)$
> $\psi_t(j) = \arg\max_i [\, \delta_{t-1}(i) \cdot a_{ij} \,]$
> $q^*_T = \arg\max_i \delta_T(i), \quad q^*_t = \psi_{t+1}( q^*_{t+1} )$

The computational cost is proportional to N^2 x T instead of N^T. Viterbi is the forward recursion with a maximum in place of a sum, plus backpointers. Because products of many probabilities underflow, all computation is executed in log space, transforming products into sums:

**Log-Space Dynamic Programming**

*φ_t(j) = max_i [ φ_{t-1}(i) + log a_ij ] + log b_j(O_t)*

# 4. Method

## 4.1 Parameter estimation

Each sentence is wrapped with an imaginary START state before the first word and an END state after the last word. We count tag-to-tag transitions (including START to first tag and last tag to END) and tag-to-word emissions on the training split. The START row of the transition table plays the role of the initial distribution π. The vocabulary is built from the training split only, and words are lowercased.

## 4.2 Smoothing

Raw counts give probability zero to anything unseen, and a single zero removes a whole Viterbi path. We therefore use add-k smoothing: a small constant is added to every legal count before normalising. Only legal moves receive probability mass (START to a tag, tag to tag, tag to END). Transitions and emissions use separate constants, tuned on the development set over the grid {0.001, 0.01, 0.1, 1}. The best values were k_trans = 1 and k_emit = 0.001. Transition smoothing hardly mattered (the table covers only 17 tags and is densely populated), while emission smoothing did.

## 4.3 Unknown words

HMM v1. Words seen fewer than twice in training are replaced by a single token, <UNK>, so the model learns what rare words look like. At test time every word outside the vocabulary maps to <UNK>.

HMM v2. The single <UNK> token is split into 11 buckets based on the shape of the word, so that, for example, a number and a capitalised name no longer share one emission distribution. The shape is read from the original casing, which keeps capital-letter information even though known words are lowercased. Each bucket learns its own emission distribution from the rare training words that fall into it.

**Table 3. Unknown-word buckets in HMM v2 (rules are applied sequentially in this order).**

| Bucket | Rule |
|---|---|
| <UNK-SYM> | No letter or digit in the word (punctuation, emoticons) |
| <UNK-URL> | Looks like a web address or e-mail (http, www, @, .com, .org, .net) |
| <UNK-NUM> | Contains a digit |
| <UNK-ALLCAP> | All capital letters, more than one character |
| <UNK-CAP> | Starts with a capital letter |
| <UNK-HYPHEN> | Contains a hyphen |
| <UNK-ING>, <UNK-ED>, <UNK-LY>, <UNK-S> | Ends in -ing, -ed, -ly or -s (with a minimum word length) |
| <UNK> | Everything else (generic fallback) |




*Figure 4. HMM v2 hierarchical morphological unknown-word classification pipeline.*

### 4.4 Decoding

Sentences are decoded with the log-space Viterbi algorithm of Section 3. The final tag must also be able to move to END, and the best path is recovered by following the backpointers. We checked the implementation on example sentences and used it unchanged for all experiments.

## 5. Experimental Setup

**Splits:** All parameters are estimated on the training split. All hyperparameter choices (smoothing constants, lowercasing, rarity threshold, bucket scheme) are made on the development split. The test split is used strictly once to report final numbers.

**Token accuracy:** The share of tokens whose predicted tag equals the gold tag.

**Macro F1:** The F1 score computed per tag over the 17 tags and averaged with equal weight, so rare tags count as much as frequent ones.

**Unknown-word accuracy:** Token accuracy restricted to tokens whose lowercased form never occurs in the training split. The same definition is used for every model so that numbers are comparable.

**Error analysis:** Per-tag precision, recall and F1, the most frequent gold-to-predicted confusions, and a confusion matrix.

## 6. Results

### 6.1 Development-set experiments

We compared lowercasing on or off, shape-based buckets on or off, the rarity threshold below which a word becomes unknown (1, 2, 3 or 5), and three values of the emission smoothing constant (the best is shown for each row). Table 4 lists every combination, sorted by development accuracy.

**Table 4. Development-set comparison of settings (best k_emit shown per row).**

| # | Lowercase | Shape buckets | min_freq | k_emit | Dev acc. | Dev unseen acc. | Note |
|---|---|---|---|---|---|---|---|
| 1 | yes | yes | 2 | 0.001 | 91.55% | 75.23% | v2 (chosen) |
| 2 | yes | yes | 3 | 0.001 | 91.30% | 74.30% | |
| 3 | no | yes | 2 | 0.001 | 90.97% | 72.54% | |
| 4 | yes | yes | 5 | 0.01 | 90.73% | 73.71% | |
| 5 | no | yes | 3 | 0.001 | 90.55% | 72.54% | |
| 6 | no | yes | 5 | 0.001 | 89.69% | 72.72% | |
| 7 | yes | no | 2 | 0.001 | 89.49% | 54.27% | v1 setting |
| 8 | no | no | 2 | 0.001 | 89.28% | 55.04% | |
| 9 | yes | no | 3 | 0.001 | 88.85% | 53.98% | |
| 10 | no | no | 3 | 0.001 | 88.53% | 55.09% | |
| 11 | yes | yes | 1 | 0.1 | 88.12% | 31.44% | |
| 12 | yes | no | 1 | 0.1 | 88.11% | 31.44% | |
| 13 | yes | no | 5 | 0.01 | 87.78% | 54.22% | |
| 14 | no | yes | 1 | 0.001 | 87.49% | 26.70% | |
| 15 | no | no | 1 | 0.001 | 87.49% | 26.70% | |
| 16 | no | no | 5 | 0.001 | 87.00% | 54.45% | |

Shape-based buckets gave the largest gain. Development accuracy rose from 89.49% (v1) to 91.55% (v2), and unknown-word accuracy from about 54% to about 75%.

Turning lowercasing off on its own did not help (89.28% against 89.49%). Splitting 'Apple' and 'apple' appears to dilute the counts of known words. Keeping lowercasing and getting the capitalisation clue through the <UNK-CAP> bucket worked best.

A rarity threshold of 1 was clearly harmful (unknown-word accuracy of 27% to 31%). With this setting no training word is ever treated as unknown, so the unknown-word rows learn nothing. A threshold of 2 was best; 3 and 5 were worse.

## 6.2 Test-set results

**Table 5. Test-set results for the two HMM versions.**

| Metric | HMM v1 | HMM v2 | Change |
|---|---|---|---|
| Token accuracy | 90.03% | 92.24% | +2.22 pp |
| Macro F1 | 83.04% | 85.53% | +2.49 pp |
| Known-word accuracy | 92.99% | 93.58% | +0.59 pp |
| Unknown-word accuracy | 53.51% | 75.72% | +22.21 pp |

HMM v2 improves every metric. Known-word accuracy changes little (92.99% to 93.58%), so almost all of the gain comes from unknown words, where accuracy rises by 22.2 points. The development gain (2.06 points) and the test gain (2.22 points) are very similar.

## 6.3 Per-tag results

**Table 6. Per-tag F1 on the test set.**

| Tag | Test tokens | F1 v1 | F1 v2 | Change |
|---|---|---|---|---|
| CCONJ | 736 | 0.993 | 0.993 | +0.000 |
| PUNCT | 3,096 | 0.987 | 0.991 | +0.004 |
| DET | 1,897 | 0.975 | 0.976 | +0.001 |
| PRON | 2,164 | 0.971 | 0.971 | +0.000 |
| AUX | 1,543 | 0.949 | 0.948 | -0.002 |
| PART | 649 | 0.939 | 0.942 | +0.003 |
| ADP | 2,025 | 0.931 | 0.936 | +0.004 |
| NUM | 542 | 0.711 | 0.918 | +0.206 |
| VERB | 2,605 | 0.893 | 0.913 | +0.020 |
| NOUN | 4,123 | 0.869 | 0.895 | +0.025 |
| ADJ | 1,788 | 0.858 | 0.878 | +0.020 |
| ADV | 1,191 | 0.857 | 0.865 | +0.008 |
| PROPN | 2,075 | 0.743 | 0.848 | +0.105 |
| INTJ | 121 | 0.834 | 0.839 | +0.005 |
| SYM | 113 | 0.791 | 0.798 | +0.008 |
| SCONJ | 384 | 0.754 | 0.763 | +0.009 |
| X | 42 | 0.062 | 0.067 | +0.005 |

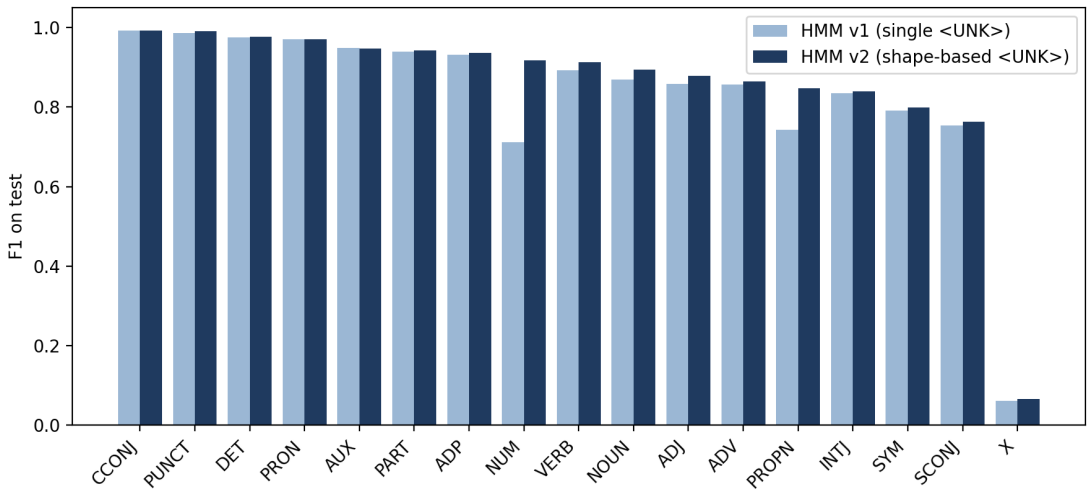



*Figure 5. Per-tag F1 on the test set for HMM v1 and v2, sorted from the easiest to the hardest tag.*

The largest gains are for NUM (F1 0.711 to 0.918) and PROPN (0.743 to 0.848), the tags that most unknown words belong to. NOUN, VERB and ADJ improve by about 0.02 to 0.03. Function-word tags such as DET, PRON, CCONJ, AUX and PUNCT were already between 0.95 and 0.99 and barely change. The tag X stays almost unsolved (F1 0.067); it has only 42 test tokens, so it barely affects accuracy but lowers macro F1.

## 6.4 Baselines, Decoding, and Emission Smoothing (Member 2 Ablations)

To provide rigorous context around the HMM tagger's performance, we implemented a baseline, compared decoding strategies, and ablated emission smoothing across multiple orders of magnitude.

**Table 7. Comparison of Most-Frequent-Tag (MFT) baseline against HMM v1 and v2.**

| Model / Baseline | Dev Accuracy | Dev Unknown Acc. | Test Accuracy | Test Unknown Acc. |
|---|---|---|---|---|
| Most-Frequent-Tag (MFT) | 86.12% | 29.63% | 86.16% | 28.59% |
| HMM v1 (Single UNK) | 89.49% | 54.27% | 90.03% | 53.51% |
| HMM v2 (Shape Buckets) | 91.55% | 75.23% | 92.24% | 75.72% |

The unigram MFT baseline assigns each word its most frequent training tag, falling back to 'NOUN' for unseen tokens. It achieves 86.16% test accuracy, showing that English lexical unigrams carry substantial information. However, MFT fails drastically on unknown words (28.59%). HMM v1 adds +3.87 pp overall (+24.92 pp unknown), and HMM v2 adds +6.08 pp overall (+47.13 pp unknown), demonstrating the crucial role of sequence transitions and morphological inductive bias.

**Table 8. Greedy decoding versus Viterbi sequence optimization.**

| Model | Decoding | Dev Accuracy | Dev Unknown Acc. | Test Accuracy | Test Unknown Acc. | Viterbi Gain (Test) |
|---|---|---|---|---|---|---|
| HMM v1 | Greedy | 87.43% | 50.88% | 87.78% | 51.97% | - |
| HMM v1 | Viterbi | 89.49% | 54.27% | 90.03% | 53.51% | +2.25 pp |
| HMM v2 | Greedy | 89.45% | 72.72% | 89.91% | 73.96% | - |
| HMM v2 | Viterbi | 91.55% | 75.23% | 92.24% | 75.72% | +2.33 pp |

Both decoders use identical transition (log A) and emission (log B) tables. Greedy decoding makes local, irreversible decisions, causing cascading errors when a locally preferred tag leads to poor downstream transitions. Viterbi dynamic programming consistently contributes +2.25 to +2.33 percentage points across both models.

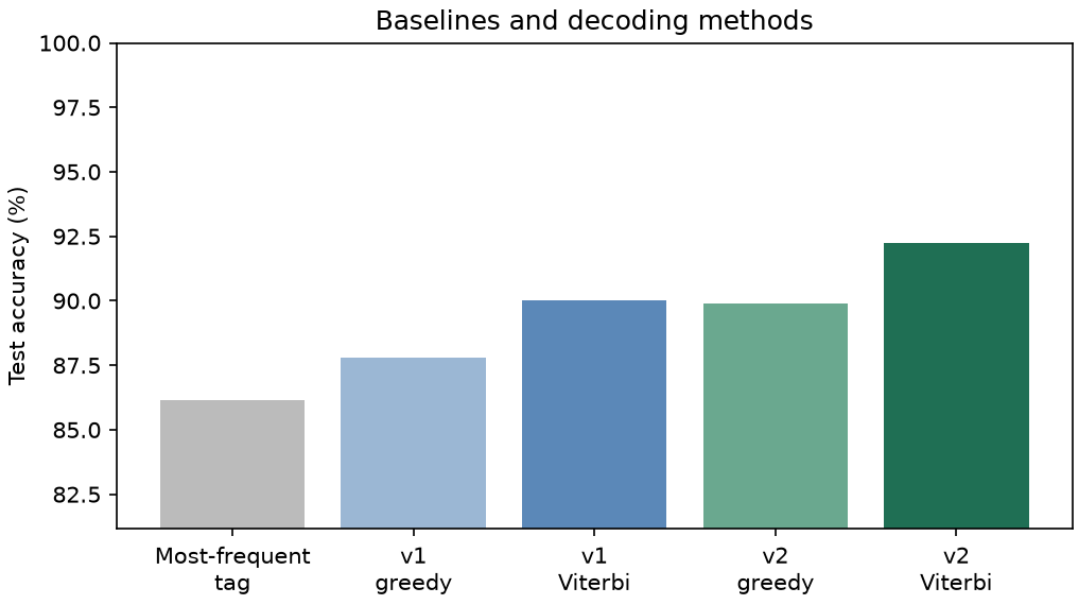



*Figure 6. Test accuracy comparison across baselines, greedy decoding, and Viterbi decoding for HMM v1 and v2.*

**Table 9. Add-k emission smoothing ablation on the development set (HMM v2).**

| k_emit | Sentences Without Valid Path | Dev Accuracy | Dev Unknown Acc. |
|---|---|---|---|
| 0 | 0 | 91.55% | 75.23% |
| 1e-06 | 0 | 91.55% | 75.23% |
| 0.001 | 0 | 91.55% | 75.23% |
| 0.1 | 0 | 91.21% | 74.65% |
| 1.0 | 0 | 88.73% | 73.42% |
| 10.0 | 0 | 80.50% | 70.20% |

Zero sentences experienced an invalid path (score of -inf) even at k_emit = 0, because morphological shape buckets map unseen words into non-empty rows. Accuracy is optimal for k <= 0.001 (91.55%) and collapses at k = 10 (80.50%). Adding large k adds k|V| pseudo-counts to the denominator, flattening the emission distribution toward uniform noise.

## 6.5 Learning Curves and Statistical Significance

**Table 10. Learning curve: dev accuracy across training data fractions.**

| Fraction | Training Sentences | HMM v1 Dev Acc. | HMM v2 Dev Acc. | v2 Advantage (Δ) |
|---|---|---|---|---|
| 10% | 1,254 | 83.28% | 87.86% | +4.58 pp |
| 25% | 3,136 | 86.41% | 89.69% | +3.28 pp |
| 50% | 6,272 | 88.44% | 90.95% | +2.51 pp |
| 100% | 12,544 | 89.49% | 91.55% | +2.06 pp |

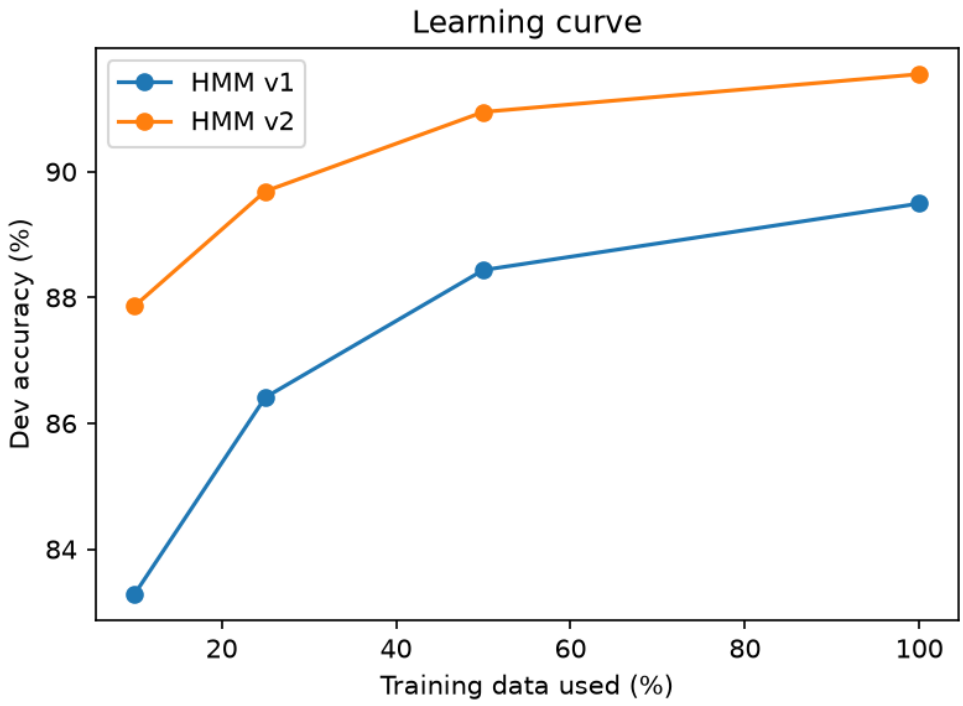



*Figure 7. Learning curve showing dev set token accuracy as a function of training data fraction (10%, 25%, 50%, 100%) for HMM v1 and HMM v2.*

Accuracy rises monotonically with data volume. Crucially, the advantage of HMM v2 over v1 is largest when data is scarce (+4.58 pp at 10% vs +2.06 pp at 100%). With limited training data, far more dev words are out-of-vocabulary, making morphological shape heuristics especially potent.

**Table 11. Non-parametric bootstrap confidence interval for (v2 - v1) on the test set (1,000 resamples).**

| Metric | Value | Statistical Interpretation |
|---|---|---|
| Mean Accuracy Difference (v2 - v1) | +2.22 percentage points (0.02217) | Substantial positive gain |
| 95% Bootstrap Confidence Interval | [+1.91%, +2.51%] | Narrow confidence bounds |
| Interval Excludes Zero | True | Statistically significant at $\alpha = 0.05$ |

By resampling test sentences with replacement 1,000 times, we confirm that the 95% confidence interval [+1.91%, +2.51%] lies strictly above zero. This proves that the improvement of HMM v2 over v1 is statistically reliable and not an artifact of random test sentence sampling.

## 7. Error Analysis

**Table 12. The ten most frequent mistakes of HMM v1 on the test set, and how often they occur in v2.**

| Gold tag → predicted tag | v1 errors | v2 errors | Change |
|---|---|---|---|
| PROPN → NOUN | 368 | 266 | -102 |
| NOUN → PROPN | 223 | 156 | -67 |
| VERB → NOUN | 120 | 106 | -14 |
| NUM → PROPN | 115 | 2 | -113 |
| VERB → AUX | 103 | 104 | +1 |
| NOUN → VERB | 96 | 84 | -12 |
| NOUN → ADJ | 85 | 89 | +4 |
| PROPN → ADJ | 75 | 34 | -41 |
| ADJ → NOUN | 72 | 69 | -3 |
| SCONJ → ADP | 64 | 63 | -1 |

The bucket scheme removes almost all NUM → PROPN errors (115 to 2) and roughly halves PROPN → ADJ (75 to 34), and it reduces PROPN → NOUN (368 to 266) and NOUN → PROPN (223 to 156). Other confusions are essentially unchanged: VERB → AUX (103 to 104), NOUN → ADJ (85 to 89), ADJ → NOUN (72 to 69) and SCONJ → ADP (64 to 63). These confusions are not tied to the shape of an unknown word, so the unknown-word buckets leave them essentially unchanged.

**Table 13. Unknown test words by gold tag (the eight most frequent), and accuracy of each model on them.**

| Gold tag | Unknown tokens | v1 accuracy | v2 accuracy | Gain |
|---|---|---|---|---|
| PROPN | 760 | 66.4% | 85.5% | +19.1 pp |
| NOUN | 538 | 59.1% | 69.7% | +10.6 pp |
| NUM | 204 | 19.1% | 97.1% | +78.0 pp |
| VERB | 153 | 61.4% | 74.5% | +13.1 pp |
| ADJ | 126 | 37.3% | 42.1% | +4.8 pp |
| X | 27 | 0.0% | 3.7% | +3.7 pp |
| PUNCT | 23 | 4.3% | 87.0% | +82.7 pp |
| ADV | 18 | 16.7% | 66.7% | +50.0 pp |

Unknown proper nouns and nouns are the most numerous unknown words. In v1, unknown numbers were tagged correctly only 19.1% of the time and unknown punctuation or symbols only 4.3% of the time, because they shared one emission row with all other unknown words. In v2 these rise to 97.1% and 87.0%. Among the larger groups, unknown nouns and adjectives remain the hardest (69.7% and 42.1%), since their shape gives few reliable clues; the tag X is also still very poor (3.7%) but covers only 27 unknown tokens.

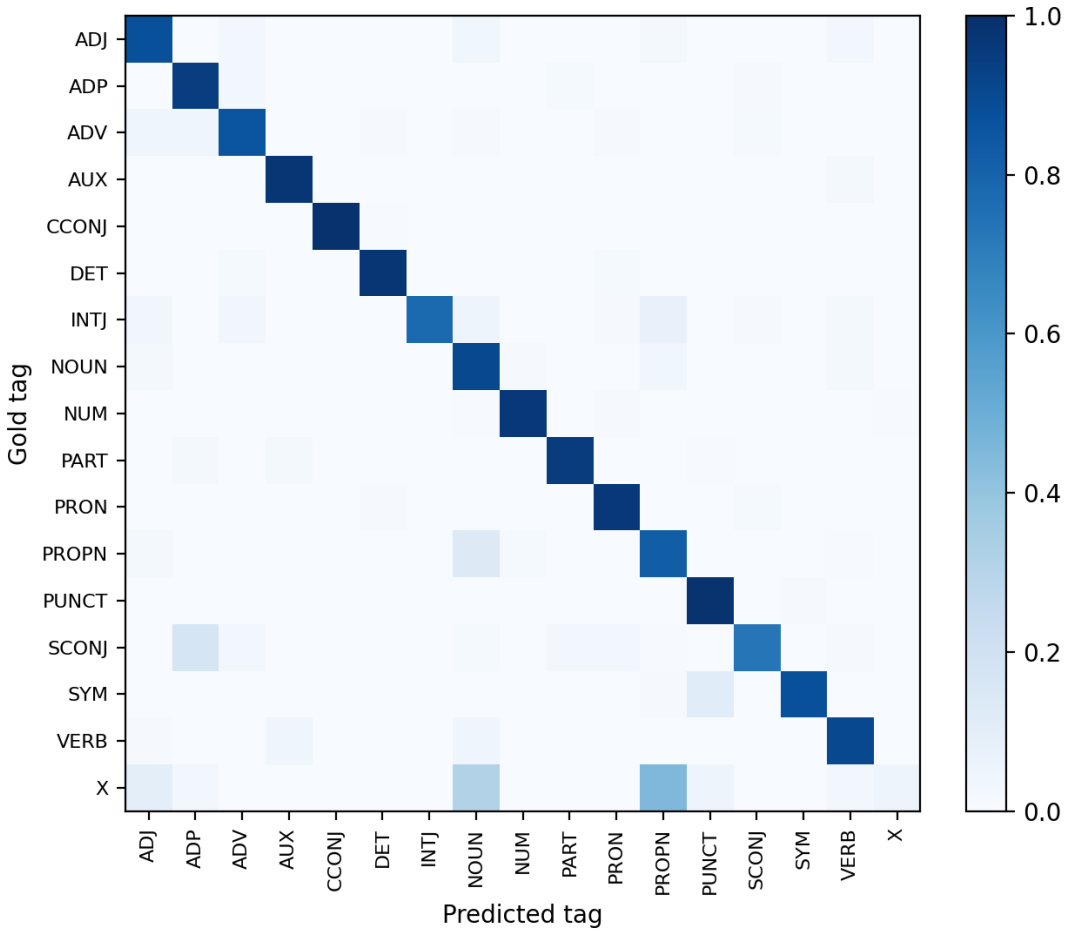



*Figure 8. Row-normalised confusion matrix of HMM v2 on the test set (rows are gold tags, columns are predicted tags).*

## 8. Decoded Sentence Examples

Tables 14 to 17 show four test sentences. In the first three, v2 fixes the mistake made by v1 (highlighted in green). Wrong predictions are shaded in red. The sentences were drawn at random with a fixed seed from sentences of 8 to 14 words.

**Table 14. Test sentence 650 (Acronym 'CDWR' corrected from ADJ to NOUN by v2).**

| Word | Gold | v1 Pred | v2 Pred |
|---|---|---|---|
| Here | ADV | ADV | ADV |
| is | AUX | AUX | AUX |
| a | DET | DET | DET |
| revised | VERB | VERB | VERB |
| draft | NOUN | NOUN | NOUN |
| of | ADP | ADP | ADP |
| the | DET | DET | DET |
| CDWR | NOUN | **ADJ** | **NOUN** |
| risk | NOUN | NOUN | NOUN |
| memo | NOUN | NOUN | NOUN |
| . | PUNCT | PUNCT | PUNCT |

**Table 15. Test sentence 484 (Plural noun 'girlfriends' corrected from VERB to NOUN by v2).**

| Word | Gold | v1 Pred | v2 Pred |
|---|---|---|---|
| I | PRON | PRON | PRON |
| am | AUX | AUX | AUX |
| bringing | VERB | VERB | VERB |
| two | NUM | NUM | NUM |
| of | ADP | ADP | ADP |
| my | PRON | PRON | PRON |
| girlfriends | NOUN | **VERB** | **NOUN** |
| from | ADP | ADP | ADP |
| LJ | PROPN | PROPN | PROPN |
| . | PUNCT | PUNCT | PUNCT |

**Table 16. Test sentence 794 (Date string '1/29/02' corrected from ADJ to NUM by v2).**

| Word | Gold | v1 Pred | v2 Pred |
|---|---|---|---|
| You | PRON | PRON | PRON |
| should | AUX | AUX | AUX |
| use | VERB | VERB | VERB |
| the | DET | DET | DET |
| same | ADJ | ADJ | ADJ |
| spreadsheet | NOUN | NOUN | NOUN |
| format | NOUN | NOUN | NOUN |
| used | VERB | VERB | VERB |
| for | ADP | ADP | ADP |
| the | DET | DET | DET |
| 1/29/02 | NUM | **ADJ** | **NUM** |
| DPR | NOUN | NOUN | NOUN |
| . | PUNCT | PUNCT | PUNCT |

**Table 17. Test sentence 230 (Auxiliary 'do' misclassified as VERB by both models).**

| Word | Gold | v1 Pred | v2 Pred |
|---|---|---|---|
| why | ADV | ADV | ADV |
| do | AUX | **VERB** | **VERB** |
| you | PRON | PRON | PRON |
| think | VERB | VERB | VERB |
| i | PRON | PRON | PRON |
| should | AUX | AUX | AUX |
| get | VERB | VERB | VERB |
| one | NUM | NUM | NUM |
| of | ADP | ADP | ADP |
| those | PRON | PRON | PRON |
| ? | PUNCT | PUNCT | PUNCT |

In the first three sentences the fixed words are unseen forms: an acronym ('CDWR', ADJ to NOUN), a plural noun ('girlfriends', VERB to NOUN) and a date ('1/29/02', ADJ to NUM). In the last sentence, 'do' is a known word, yet it is tagged VERB instead of AUX. Smoothing and unknown-word buckets cannot fix this kind of error, because the HMM only sees the previous tag.

# 9. Discussion

## 9.1 The gap: why a BiLSTM next

HMM v2 closes much of the unknown-word gap, but its structure still limits it.

**Limited context:** The tag of a word depends only on the previous tag, and each word depends only on its own tag. Rabiner (1989, Section VIII-A) notes that the Markov assumption is inappropriate when dependencies extend over several states. Remaining errors such as VERB versus AUX or NOUN versus PROPN are the kind of ambiguity that wider context can help to resolve, although we did not measure long-range dependencies directly.

**Unknown words handled by hand-written rules:** The buckets use a small fixed list of shapes and suffixes. A BiLSTM with character or subword embeddings can learn such cues itself, and can combine them with the surrounding sentence.

**Left-to-right scoring:** The HMM scores a tag from its left neighbour through the transition table, while a bidirectional model sees context on both sides of a word.

## 9.2 Limitations

**Test-set exposure:** The idea of shape-based buckets came from the error analysis of v1, which was carried out on the test set. All choices among settings were made on the development set, and the development and test gains are similar (about 2.1 and 2.2 points), but the test result of v2 may still be slightly optimistic.

**Domain and language specificity:** Hand-written bucket rules reflect English spelling and the style of this corpus, and may transfer poorly to other languages or domains.

**Rare tags:** The tag X (42 test tokens) is nearly unsolved and has a large effect on macro F1 relative to its effect on accuracy.

**Single split:** All results come from the official train, development and test split. We did not repeat the experiments on other splits.

**Overlap between splits:** A few very short sentences appear in both train and test (Section 2.1). Leaving them out changes HMM v2 test accuracy by 0.03 points (92.24% to 92.21%), so this does not change our conclusions.

# 10. Conclusion and Next Steps

We built an HMM POS tagger with add-k smoothing and log-space Viterbi decoding. The baseline reaches 90.03% test accuracy; shape-based unknown-word buckets raise this to 92.24%, with macro F1 of 85.53% and unknown-word accuracy of 75.72%. The remaining errors are concentrated in ambiguities between categories that share word forms, and in unknown nouns and adjectives.

The next phase will train a BiLSTM tagger with word and character or subword embeddings, masking padded positions in the loss (Huang, Xu and Yu, 2015). For a fair comparison it must use the same data files, the same 17 tags, the same unknown-word definition and the same metrics. The HMM v2 numbers in Table 5 are the reference to compare against.

## Appendix A. Reproducibility and Hand-off

**Table 18. Final settings of the two models.**

| Setting | HMM v1 | HMM v2 |
|---|---|---|
| Lowercase words | yes | yes |
| Rarity threshold (min_freq) | 2 | 2 |
| Unknown-word handling | single <UNK> | 11 shape buckets |
| k_trans | 1.0 | 1.0 |
| k_emit | 0.001 | 0.001 |
| Vocabulary size | 8,866 | 8,876 |

The experiments were run in Python with NumPy, conllu, scikit-learn and Matplotlib, in a Colab notebook and standalone modular package. Two files are saved for the next phase: hmm_results.json (settings, test metrics and per-tag F1) and hmm_test_predictions.json (words, gold tags and HMM v2 predictions for every test sentence).

**Checklist for the DL phase:** Use the same three EWT files (UD v2.18), skipping multiword tokens and empty nodes; the same 17 tags in alphabetical order (ADJ, ADP, ADV, AUX, CCONJ, DET, INTJ, NOUN, NUM, PART, PRON, PROPN, PUNCT, SCONJ, SYM, VERB, X); a vocabulary built from the training split only; the unknown-word definition of Section 5; the metrics of Section 5, with padded tokens masked out of the loss and out of every metric; hyperparameters chosen on the development split, with the test split used once.

## References

Rabiner, L. R. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. Proceedings of the IEEE, 77(2), 257-286.

Huang, Z., Xu, W., and Yu, K. (2015). Bidirectional LSTM-CRF models for sequence tagging. arXiv:1508.01991.

Universal Dependencies English EWT treebank, release v2.18 (CC BY-SA 4.0). https://github.com/UniversalDependencies/UD_English-EWT